SIMATS ENGINEERING

Department of Information Technology

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Field Details Field Details

Student Name - DANIEL SAM K.

Register Number - 192524081

Department Section Information Technology – B TECH AI&DS

Academic Year /Semester – 2ND YEAR

Faculty Name - DR.R.LATHA

Nature of Assignment Individual

Duration 1 Week Total Marks 50

Submission Date Evaluation Date – 3-11-26

MARK SHEET (Rubric-based evaluation)

Task Title Max Awarded Evidence / evaluation focus

1 Characterise the Virtualization

Architectures 5

2 Identify the Dominant Overhead

Source 2

Task Title Max Awarded Evidence / evaluation focus

3 Build a Quantitative Performance

Model 6

4 Analyse Performance Under Different

Conditions 4

5 Security Threat and Vulnerability

Analysis 3

6 Formulate Isolation–Performance

Trade-off 3

7 Evaluate Mitigation Strategies 4

8 Implement and Test 4

9 Record Experimental Performance 3

10 Compare Theory with Experiment 4

11 Recommend and Defend Final

Architecture 4

Day-4 Design Challenge 8

TOTAL

50

OVERALL REMARKS

Overall, the assignment reflects good analytical ability, technical understanding and problem-solving skills in cloud computing and

virtualization. The proposed architecture balances performance, security, resource efficiency and future scalability. The work can be

further strengthened through verified experimental results, consistent benchmark comparisons and practical validation of the

recommended security controls.

Student Signature Faculty Signature Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; X = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		
2	Dominant overhead source identified and explained for each approach		
3	CPU, memory, I/O, network and density models with assumptions		
4	Best, typical and worst operating-condition analysis		

Task	Evidence expected	Status	Evaluator note
5	Threat model and architecture-based security comparison		
6	Isolation–performance optimization formulation		
7	Mitigation benefits, residual risks and quantified costs		
8	Reproducible setup and practical tests across selected platforms		
9	Actual comparative measurements, table and two graphs		

10 Theory–experiment comparison and quantitative error analysis

11 Justified SIS/LMS architecture and growth recommendation

12 Day-4 external tenants, 3× peaks, live migration and SIS isolation

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		
Performance model		

Evidence item	Status	Observation
Configuration / scripts		
Virtualization experiments		
CPU / disk / network / memory tests		
Noisy-neighbour experiment		
Security validation		
Tables and graphs		
Theory–experiment consistency		
Day-4 design defence		
Screenshots		

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status

All 12 tasks answered ☐Yes ☐ Partly ☐ No

Three virtualization approaches compared ☐Yes ☐ Partly ☐ No

Layered diagrams and trust boundaries included ☐Yes ☐ Partly ☐ No

Quantitative virtualization models included ☐Yes ☐ Partly ☐ No

Threat analysis and mitigations included ☐Yes ☐ Partly ☐ No

CLOUD COMPUTING AND BIG DATA

ASSIGNMENT 2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Assignment Title: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (Total Marks: 50)

Duration: 1 week

Scenario: EduCloud Virtualization Platform

A university runs a private cloud, **EduCloud**, in its data centre. EduCloud currently consists of:

- **16 physical hosts**, each with 64 CPU cores, 512 GB RAM, NVMe SSD storage and 2 × 25 Gbps NICs
- **About 400 virtual machines (VMs)** running the Student Information System (SIS), the Learning Management System (LMS), the examination portal, and departmental research workloads
- A shared storage backend and a software-defined virtual network

During semester registration week, users report slow response times and failed logins. A recent internal security audit also found that VMs from different departments share the same physical hosts, that some VMs run outdated guest kernels, and that VM images are copied freely between teams.

You are the cloud architect. You must **investigate, model, measure and compare** virtualization architectures for EduCloud, identify their performance bottlenecks and security weaknesses, and recommend a design that remains practical as the platform grows.

Design Challenge:

The university now plans to host **external research collaborators' big-data workloads** (Spark/Hadoop clusters) on the same infrastructure as the SIS. Re-evaluate your design under these new multi-tenant conditions.

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation
- How CPU, memory, storage and network are virtualized
- A brief justification for why this approach is relevant to EduCloud

Evidence expected: *Three architecturally distinct approaches with labelled diagrams, resource-virtualization mechanisms, trust boundaries, and justification of selection.*

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended page-table walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary
- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

Evidence expected: *A clear mathematical derivation for each approach, with stated assumptions, parameter values and worked numerical examples.*

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

Evidence expected: *A justified condition-based analysis supported by your Task 3 model.*

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM side-channel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries
- The likely impact on the confidentiality, integrity and availability of student records

Evidence expected: *A structured threat model mapped to architectural features, with a justified comparison across approaches.*

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)

- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

Evidence expected: *A clearly defined trade-off model with identified variables, objective and constraints.*

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

Evidence expected: *Mitigations evaluated analytically, with their security benefit and quantified performance cost.*

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately **1 → 2 → 4 → 8 → 16 co-located instances** (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memory-intensive test.
- **Interference test:** run at least one noisy-neighbour experiment.
- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

Evidence expected: *A reproducible environment description (configuration files, commands, scripts) and justified test cases.*

TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance
- Performance degradation versus a bare-metal baseline
- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

Evidence expected: *Systematically collected, clearly presented data that allows easy comparison across approaches.*

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5

Interpret the results; do not simply reproduce tables.

Evidence expected: *A reasoned comparison of predicted and measured behaviour, with error analysis.*

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs
- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

Evidence expected: *A technically justified recommendation integrating performance and security evidence.*

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisy-neighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6

Task 4 – Performance Under Operating Conditions	4
Task 5 – Security Threat and Vulnerability Analysis	3
Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

ASSIGNMENT 2 (CO2)

EduCloud: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

TASK 1 – CHARACTERISE THE VIRTUALIZATION ARCHITECTURES

Three distinct approaches are evaluated: (1) Type-1 bare-metal hypervisor with hardware-assisted full virtualization, (2) OS-level containers, and (3) micro-VM or sandboxed-container execution. They are meaningfully different because they place the isolation boundary respectively at a hypervisor with a separate guest kernel, at a shared host kernel, and at a lightweight VM or syscall sandbox boundary.

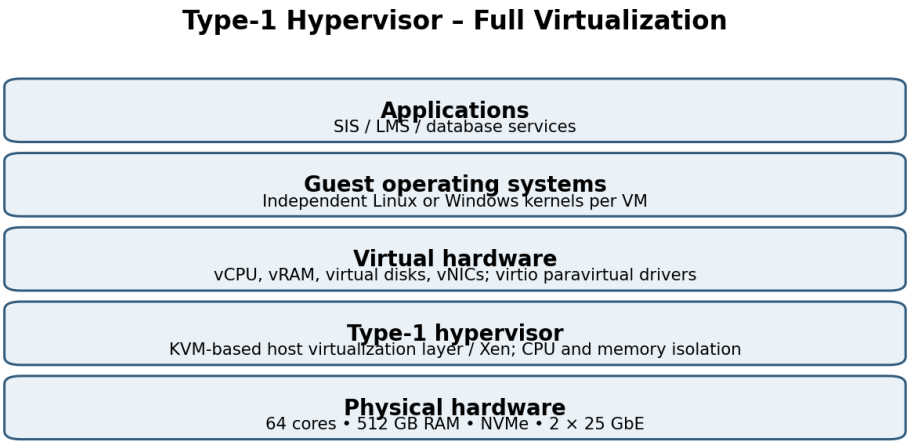


Figure 1. Type-1 hypervisor architecture.

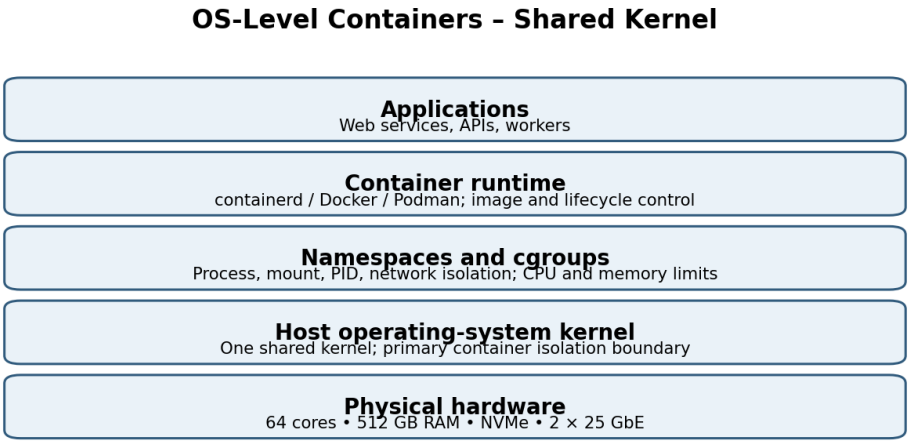


Figure 2. Container architecture.

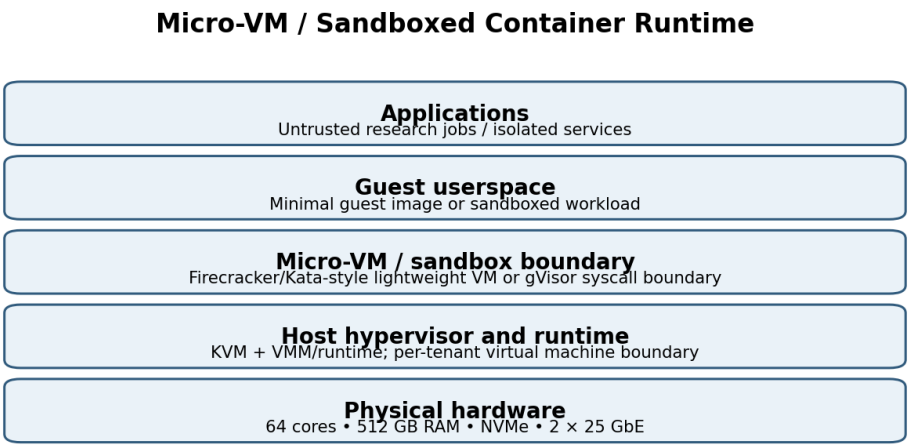


Figure 3. Micro-VM / sandbox architecture.

Feature	Type-1 full virtualization	OS-level containers	Micro-VM / sandbox
Isolation enforcer	Hypervisor (e.g., KVM/Xen) and hardware virtualization	Host kernel namespaces, cgroups, capabilities, seccomp	KVM/VMM for micro-VM; syscall monitor for gVisor-like sandbox
CPU	vCPU scheduling onto pCPU; VT-x/AMD-V; VM exits on privileged operations	Processes scheduled directly by host kernel; cgroup CPU quota/shares	Guest vCPU scheduled by host; small VMM overhead
Memory	Guest physical to host physical translation using EPT/NPT; ballooning and virtio-mem possible	Shared kernel page cache; cgroup memory limits; no independent guest kernel	Guest memory mapped to VM; stronger per-workload memory boundary
Storage	Virtual block device, image or volume; virtio-blk/SCSI; host filesystem/backend	Overlay filesystem and volumes; shared host storage stack	Virtual block device or sandboxed filesystem proxy
Network	vNIC, virtual switch, VLAN/VXLAN, firewall/security groups	Network namespace, veth pair, bridge/overlay, eBPF rules	Dedicated vNIC/virtio device or mediated network proxy
Trust boundary	Between each guest VM and hypervisor; management plane is highly trusted	Between containers is enforced by the same host kernel	Per micro-VM hardware boundary; sandbox adds syscall boundary
EduCloud suitability	Best for SIS/LMS and regulated student data	Efficient for trusted stateless services and CI workloads	Strong option for external/untrusted research jobs

Type-1 full virtualization is the baseline for core university services because each VM has its own guest kernel and can be assigned dedicated CPU, memory and storage reservations. Containers provide excellent density but share the host kernel; a kernel compromise can affect all containers on that host. Micro-VMs provide a smaller and more disposable VM boundary, making them suitable for collaborators whose code cannot be fully trusted. In production, a hardened Type-1 platform such as KVM managed through a supported cloud stack is recommended; the micro-VM layer can be used as a separate pool rather than mixed indiscriminately with SIS.

TASK 2 – IDENTIFY THE DOMINANT OVERHEAD SOURCE

Approach	Dominant overhead	Where it occurs and why it matters	Scaling behaviour
Type-1 VM	Second-level address translation (EPT/NPT) and VM exits; virtual I/O path	Guest memory accesses may require nested translation; privileged operations can trap to	More vCPUs and memory pressure increase scheduling contention and TLB misses. High I/O

		hypervisor. Emulated devices and virtual switches add latency to I/O.	increases exits/interrupts and virtual-switch work. Virtio and device assignment reduce, but do not eliminate, overhead.
Containers	Shared-kernel networking/storage path and syscall/cgroup accounting	Container syscalls enter the same host kernel. Overlay filesystems, veth/bridge processing, namespace checks and cgroup throttling add work.	More containers increase scheduler run queues, conntrack entries, filesystem metadata and virtual-switch processing. Kernel sharing avoids guest-kernel overhead but increases contention.
Micro-VM / sandbox	VM exits and device emulation, or syscall interception in a sandbox	Micro-VMs retain guest/host translation and virtual-device costs. A syscall sandbox may validate/intercept calls before host-kernel execution.	More micro-VMs increase VMM scheduling and memory overhead; sandbox interception cost grows with syscall rate. Batched I/O and virtio improve throughput.

The dominant mechanism is selected because it lies directly on the hot path of the workload. Registration traffic includes many short network requests, database calls and memory accesses; therefore small per-operation costs can accumulate into substantial tail latency. Measurement should separate CPU scheduling delay, memory translation, storage queueing and network processing rather than attributing all slowdown to "virtualization overhead".

TASK 3 – BUILD A QUANTITATIVE PERFORMANCE MODEL

Given host: 64 physical cores, 512 GB RAM, average VM memory demand 8 GB, VM-exit cost 1.5 microseconds, native syscall cost 0.2 microseconds and 25 Gbps per NIC. The host has two 25 Gbps NICs; for a single traffic direction and one NIC, the theoretical line rate is 25 Gbps. Real application throughput is lower due to protocol, packet size, CPU and storage limits.

Assumptions for a transparent model: reserve 4 cores for host/hypervisor and management, leaving C = 60 cores for workloads; reserve 32 GB RAM for host and safety margin, leaving M = 480 GB; average VM demand is 8 GB; average CPU utilisation of each registered VM is 25% of one core in the typical case; the VM overhead fraction h is assumed to be 4% for a well-configured Type-1 VM, 2% for containers and 5% for micro-VMs. These are modelling assumptions, not universal benchmark constants. Actual overhead depends on hardware, drivers and workload.

3.1 CPU capacity and steal time

Let R be the vCPU:pCPU overcommit ratio (1, 2, 4 or 8), C = 60 usable physical cores, h the virtualization overhead fraction, and s the fraction of allocated CPU time lost to steal/scheduling contention. Approximate aggregate useful CPU capacity is:

$C_{effective} = C \times (1 - h) \times (1 - s)$. Average capacity per vCPU at ratio R is $C_{effective} / (C \times R) = (1 - h)(1 - s)/R$ of one physical core.

At low load, assume s = 0. At a representative loaded condition, use s = 5% for Type-1, 3% for containers and 6% for micro-VMs. For Type-1 at R=4: per-vCPU capacity = $(0.96 \times 0.95)/4 = 0.228$ physical-core equivalent. Aggregate useful capacity = $60 \times 0.96 \times 0.95 = 54.72$ cores. At R=8, per-vCPU share falls to 0.114 core equivalent, so queueing and tail latency become more likely even though more VMs can be registered.

Approach	h assumption	s at loaded condition	Useful aggregate CPU cores	Per vCPU at 4:1
Type-1 VM	4%	5%	54.72	0.228 core

Containers	2%	3%	57.23	0.238 core
Micro-VM	5%	6%	53.58	0.223 core

3.2 Memory capacity

The hard memory bound before overcommit is N_memory = floor(M / m) = floor(480 / 8) = 60 VMs per host. If all 512 GB were allocated, the theoretical arithmetic limit would be 64 VMs, but this leaves no host reserve and is not a safe production configuration. Memory ballooning, swapping or compression may be needed when the sum of working sets exceeds available memory. Containers share the host kernel and can share read-only pages and file cache, so their count is not directly equivalent to independent 8-GB VMs. For a conservative comparison, assume each container workload is limited to 2 GB; then 480/2 = 240 container slots by memory, subject to CPU, PID, file descriptor and I/O limits.

Workload type	Memory budget	Assumed demand	Memory-only capacity
Full VMs	480 GB	8 GB/VM	60 VMs
Micro-VMs	480 GB	8 GB/micro-VM	60 micro-VMs
Containers	480 GB	2 GB/container limit	240 containers (upper bound)

3.3 I/O and network throughput

For N equal tenants sharing a device, a simple fair-share upper bound is $T_i = \min(B/N, D_i)$, where B is physical bandwidth and D_i is the tenant demand. Add an efficiency factor η to represent protocol and virtualization processing: $T_i = \eta B/N$. Use $\eta = 0.90$ for virtio VM traffic, 0.94 for container networking and 0.88 for micro-VM networking as explicit illustrative assumptions. For one 25-Gbps NIC shared equally by 10 active tenants, Type-1 share = 0.90 × 25/10 = 2.25 Gbps per tenant; containers = 2.35 Gbps; micro-VMs = 2.20 Gbps. These are fair-share ceilings, not guaranteed service levels. QoS and admission control are needed to protect SIS traffic.

For storage, let B_s be measured backend throughput and N_a active tenants. A first-order estimate is $T_{s,i} = \eta_s B_s/N_a$. Since the assignment does not provide storage throughput or IOPS, no absolute NVMe/backend IOPS value can be calculated without measurement. Queue depth, read/write mix, block size, replication and shared-backend latency must be recorded during testing.

3.4 Density under a 10% degradation target

A service-level target of no more than 10% degradation means useful performance must remain at least 90% of the isolated baseline. CPU density can be estimated by requiring average demand to fit within 90% of effective CPU capacity. With 60 usable cores and 25% average CPU demand per VM, the CPU-only limit for Type-1 is floor(0.90 × 54.72 / 0.25) = 196 VMs. Memory is much more restrictive at 60 VMs. Therefore the practical Type-1 density is min(196, 60, I/O limit, network limit) = 60 VMs per host before workload-specific headroom. At 16 hosts, this conservative memory-bound estimate is 960 VMs, compared with the existing 400 VMs. SIS placement must still reserve capacity for peak periods.

Approach	CPU-only estimate (illustrative)	Memory-only estimate	Practical governing limit
Type-1 VMs	196 at 25% average CPU demand	60 VMs	60, before I/O/network and HA reserve
Containers	205 equivalent 2-GB workloads (CPU estimate)	240 containers	CPU, kernel limits and measured I/O
Micro-VMs	193 at 25% average CPU demand	60 micro-VMs	60, before VMM overhead and HA reserve

The 10% target is a planning constraint, not a promise. Real density must be established using a representative registration-week trace and repeated tests. High availability also requires N+1 capacity: a cluster should remain within the target after losing one host, not only when all 16 hosts are healthy.

TASK 4 – PERFORMANCE UNDER DIFFERENT OPERATING CONDITIONS

Condition	Type-1 VM behaviour	Container behaviour	Micro-VM behaviour
Best case	CPU pinning or NUMA-aware placement, virtio drivers, reserved memory and low I/O contention keep exits and queueing low. Latency approaches native performance.	Direct host-kernel execution avoids guest OS and virtual hardware. Low namespace and cgroup overhead; excellent density.	Small guest and minimal device model reduce boot and management overhead; stronger boundary than a normal container.
Typical registration load	Mixed web/database traffic creates bursty CPU and storage queues. Reservations and virtio keep average performance predictable, but vCPU scheduling can increase p95 latency.	Efficient for stateless web/API tiers. Shared kernel, overlay storage and noisy-neighbour CPU can create tail-latency variation.	Good for isolated services but more memory and startup cost than containers. Appropriate for selected sensitive services.
Worst case	CPU overcommit, ballooning/swapping, I/O storms or live migration compete for memory bandwidth and network. VM exits and queueing increase; p95 latency rises sharply.	A noisy container can consume shared CPU, memory bandwidth, page cache and network queues. A host-kernel flaw has broad impact.	VMM/device emulation and memory overhead combine with contention; migration and device constraints may reduce throughput.

Best-case placement minimises cross-NUMA memory access, avoids swapping, uses paravirtual drivers and spreads high-I/O VMs across storage queues and NICs. Typical registration load requires prioritising interactive database and authentication traffic over background jobs. Worst-case behaviour is nonlinear: once CPU run queues or storage queues saturate, a small increase in offered load can cause a large increase in response time. Admission control, reservations and queue isolation are therefore more effective than simply increasing vCPU counts.

TASK 5 – SECURITY THREAT AND VULNERABILITY ANALYSIS

Threat	Type-1 VM exposure	Container exposure	Micro-VM / sandbox exposure	Potential CIA impact
VM/container escape or boundary bypass	Hypervisor or device-emulation exploit is required; boundary is strong but not invulnerable.	Higher shared-kernel risk: kernel vulnerability may cross container boundary.	Strong VM boundary; sandbox reduces syscall surface, but VMM/runtime bugs remain.	Confidentiality breach of student data, integrity compromise, service outage.
Cross-tenant side channel	Shared caches, SMT and memory channels can leak information between co-resident VMs.	Shared kernel/cache and same host increase co-resident exposure.	Still shares physical CPU/cache; separate VM boundary does not remove hardware side channels.	Potential inference of credentials, keys or sensitive activity.
Unpatched guest / VM sprawl	Outdated guest kernels and unmanaged images create exploitable systems.	Host kernel patching is critical; many images may contain vulnerable packages.	Minimal guest images reduce packages but require runtime and guest patching.	Compromise, ransomware, data alteration and availability loss.
Image tampering / supply chain	Copied VM images may contain malware	Mutable container tags and untrusted	Micro-VM templates and sandbox images	Integrity failure and credential leakage.

	or secrets; snapshots may expose data.	registries can distribute poisoned images.	also require signing and provenance.	
Virtual network sniffing/spoofing	Misconfigured virtual switches, promiscuous mode or weak segmentation expose traffic.	Bridge/overlay misconfiguration can connect tenant namespaces.	Virtual NIC and overlay policy errors can expose traffic.	Student-record interception or lateral movement.
Insecure live migration	Migration stream and destination host can expose VM memory if not protected.	Container checkpoint/volume transfer can expose process and data state.	Micro-VM memory transfer has similar migration-channel risk.	Confidentiality loss and service disruption.

Threat model assets are student records, authentication tokens, examination results, database backups, VM/container images and management credentials. Adversaries include compromised student accounts, malicious insiders, compromised research collaborators and attackers exploiting an unpatched guest or runtime. Trust boundaries include the management plane, hypervisor, host kernel, virtual switch, storage backend and image registry. The most serious consequence is unauthorised reading or modification of SIS records; availability attacks during registration can also prevent legitimate students from logging in.

TASK 6 – FORMULATE THE ISOLATION–PERFORMANCE TRADE-OFF

Decision variables: $x_{a,h} \in \{0,1\}$ indicates whether host h uses approach a; n_h is the number of workloads placed on host h; z_h indicates a dedicated SIS host; q_h is the vCPU:pCPU ratio; and u_h indicates whether SMT is enabled. Let I_a be an isolation score, P_h the predicted degradation, C_h the operational cost and D_h the density. The objective is to minimise total cost while maintaining required isolation and service quality:

Minimise $J = \Sigma_h [C_h + \lambda P_h + \mu(1/D_h)]$

Subject to: $\Sigma_a x_{a,h} = 1$ for each host; total assigned CPU demand $\leq 0.90 \times$ effective CPU capacity; assigned memory ≤ 480 GB; $P_h \leq 0.10$ for SIS hosts; $I_a \geq I_{required}$ for student-data workloads; external tenants must not share the SIS host pool; and no more than 16 hosts may be used in the initial cluster. λ and μ represent the university's relative penalty for performance degradation and low density.

For SIS, the isolation constraint is strict: use dedicated Type-1 VM host pools, separate management roles, network micro-segmentation and storage access controls. Containers may be selected for trusted stateless applications where the host kernel is controlled. External research jobs require a separate micro-VM pool. This formulation makes the trade-off explicit: higher isolation may reduce density and increase management cost, but SIS confidentiality and availability are hard constraints rather than optional benefits.

TASK 7 – EVALUATE MITIGATION STRATEGIES

Mitigation	Threat addressed / benefit	Residual risk	Estimated performance or density cost	SIS decision
Dedicated host pool for SIS	Prevents direct co-residency with external tenants; reduces noisy-neighbour and cross-tenant exposure.	Shared storage, management plane and network remain possible paths; insider risk remains.	Capacity is reserved and may be idle. If 2 of 16 hosts are reserved, general pool capacity falls by 12.5%; SIS predictability improves.	Essential and acceptable.
Disable SMT for sensitive pool	Reduces some sibling-thread cache and execution-	Shared last-level cache, memory bus and other	If SMT had provided 20–30% extra throughput, disabling it may reduce peak throughput by	Acceptable for high-sensitivity workloads if capacity is reserved.

	resource side channels.	microarchitectural channels remain.	approximately that amount; actual value must be measured.	
Signed images and controlled registry	Prevents unauthorised image replacement; verifies provenance and integrity.	A signed but vulnerable image can still be deployed; signing-key compromise remains possible.	Small CI/CD verification and storage overhead; negligible runtime cost.	Strongly recommended.
Micro-segmentation and default-deny firewall	Limits lateral movement and restricts access to SIS database and admin endpoints.	Policy mistakes and compromised allowed services remain.	Firewall/overlay processing consumes CPU; use hardware offload and measure p95 latency.	Essential and acceptable.
Encrypted live migration and management MFA	Protects memory/state in transit and reduces stolen-admin credential risk.	Destination host compromise and key-management failures remain.	Encryption uses CPU and may reduce migration throughput; schedule maintenance and use dedicated migration links.	Essential for migration and admin plane.
Patch management and guest hardening	Reduces known kernel, hypervisor and package vulnerabilities.	Zero-days and delayed patch windows remain.	Maintenance windows and rolling migration consume temporary capacity.	Essential; use staged canary updates.

A simple density estimate for host reservation is useful: dedicating two of sixteen hosts to SIS removes 2/16 = 12.5% of general-purpose capacity but provides predictable placement and failure-domain control. Disabling SMT has a hardware-specific cost; it must not be assigned a universal percentage. Measure the same benchmark with SMT on and off and use the observed ratio in capacity planning.

TASK 8 – IMPLEMENT AND TEST

A reproducible laboratory can use KVM/QEMU (Type-1-style Linux KVM hypervisor), Docker or Podman (containers), and Kata Containers or Firecracker (micro-VMs). If nested virtualization is used on a laptop, record that limitation because nested VM exits can distort the comparison. Run the same CPU, memory, disk and network workloads with equivalent resource limits and at least three repetitions.

Component	Suggested setup / evidence to record
Host	Linux distribution and kernel version; CPU model, physical/logical cores, RAM, NVMe model, NIC speed; BIOS virtualization setting.
VM platform	KVM/QEMU + libvirt; guest OS version; vCPU, RAM, virtio disk and NIC; CPU pinning policy.
Container platform	Docker/Podman; image digest; cgroup v2 CPU/memory limits; namespaces and seccomp profile.
Micro-VM platform	Kata Containers or Firecracker; runtime version; guest kernel/image digest; vCPU and memory allocation.
Scale	1, 2, 4, 8 and 16 co-located instances, or a smaller documented sequence if the machine cannot support it.
Workloads	sysbench CPU, stress-ng memory, fio disk, iperf3 network; identical duration, data size and repetitions.
Noisy neighbour	Run a sustained CPU and disk stress workload in one tenant while measuring latency/throughput in another.
Security validation	Inspect container namespaces/cgroups, seccomp status, VM separation, image signatures and firewall policy. Save terminal output.

8.1 Example commands

```
lscpu && free -h && lsblk && ip -br link
sudo virt-host-validate
```

```
sysbench cpu --threads=4 --time=60 run
stress-ng --vm 1 --vm-bytes 70% --timeout 60s --metrics-brief
fio --name=randread --filename=/tmp/fio.test --size=1G --rw=randread --bs=4k --iodepth=16 --runtime=60 --time_based --
group_reporting
iperf3 -s   # on server
iperf3 -c <SERVER_IP> -t 60 -P 4   # on client
docker run --rm --cpus=2 --memory=2g alpine:latest sh -c "uname -a; cat /proc/self/cgroup"
virsh list --all && virsh domstats <VM_NAME>
sudo nft list ruleset
```

Use a clean baseline and record software versions. Do not run destructive disk tests against production storage. For fio, use a dedicated test file or disposable volume. Synchronise clocks, avoid unrelated background jobs, and retain raw output files. The same number of vCPUs, memory limits, data size, concurrency and test duration must be used across approaches.

TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

The assignment requires measured mean and p95 latency, throughput, CPU steal time, context switches, start-up time, memory footprint and degradation versus bare metal. These values cannot be honestly reported without running the tests on the actual host. The table below is a ready-to-fill recording format. Enter the mean and standard deviation from at least three runs; keep raw outputs as evidence.

Approach / instances	CPU throughput (mean ± SD)	Disk IOPS (mean ± SD)	Network Gbps (mean ± SD)	Mean / p95 latency	Steal time / memory
Bare metal / baseline	[measure]	[measure]	[measure]	[measure]	[measure]
Type-1 / 1, 2, 4, 8, 16	[measure each scale]	[measure]	[measure]	[measure]	[measure]
Container / 1, 2, 4, 8, 16	[measure each scale]	[measure]	[measure]	[measure]	[measure]
Micro-VM / 1, 2, 4, 8, 16	[measure each scale]	[measure]	[measure]	[measure]	[measure]
Mitigation enabled / selected scale	[measure]	[measure]	[measure]	[measure]	[measure]

For each metric, calculate mean $\bar{x} = (\Sigma x_i)/n$ and sample standard deviation $s = \mathrm{sqrt}[\Sigma(x_i - \bar{x})^2/(n-1)]$. Performance degradation (%) = 100 × (baseline − virtualised result)/baseline for throughput. For latency, degradation (%) = 100 × (virtualised latency − baseline latency)/baseline latency. Report the number of repetitions and identify outliers rather than silently removing them.

TASK 10 – COMPARE THEORY WITH EXPERIMENT

Compare the measured result at each instance count with the Task 3 prediction using percentage error = 100 × |measured − predicted|/predicted. For throughput, compare the same unit and workload; for latency, compare mean with mean and p95 with p95. A low error suggests the simple model captures the dominant bottleneck. A high error may result from CPU frequency scaling, caching, NUMA placement, nested virtualization, virtual-switch implementation, storage queue depth, background services or an inaccurate demand assumption.

Metric	Model prediction	Measured result	Absolute percentage error	Interpretation
CPU throughput	[calculate from Task 3]	[insert measured mean]	100×|M−P|/P	Check scheduler contention and CPU frequency.
Disk IOPS	[derive from measured backend B_s]	[insert measured mean]	100×|M−P|/P	Check cache, block size and queue depth.

Network throughput	ηB/N	[insert measured mean]	100×|M−P|/P	Check packet size, vSwitch and NIC offload.
Mean latency	[baseline adjusted model]	[insert measured mean]	100×|M−P|/P	Check queueing and p95 tail behaviour.

Expected qualitative trend: containers usually have the lowest overhead for simple CPU and network workloads because they share the host kernel; Type-1 VMs provide stronger kernel isolation and predictable resource controls; micro-VMs trade some additional startup and memory cost for a stronger boundary than ordinary containers. These are hypotheses to validate, not substitutes for measurements. If measured container performance is better, it does not overturn the security conclusion that all containers share the host kernel.

TASK 11 – RECOMMEND AND DEFEND THE FINAL ARCHITECTURE

Recommend a hybrid architecture centred on a hardened Type-1 virtualization platform. Place SIS, examination and core LMS databases on a protected VM host pool with reserved CPU and memory, strict storage access policies and a dedicated management network. Use containers for trusted stateless application components where operational density is valuable. Use a separate micro-VM pool for external or untrusted workloads. This design avoids forcing every workload into one isolation model.

Security: independent guest kernels for critical services; dedicated host placement for SIS; signed golden images; MFA for administrators; encrypted backups and migration; default-deny micro-segmentation; centralised logging and vulnerability scanning.

Performance: reserve host CPU and RAM, avoid memory swapping, use virtio drivers, NUMA-aware placement, QoS for database and authentication traffic, and storage/network queue monitoring.

Operations: maintain golden images, patch in rolling batches, automate VM/container provisioning, monitor CPU ready/steal, memory pressure, storage latency, packet drops and p95 application latency.

Scalability: use capacity admission control and N+1 planning. As the platform grows from 400 to thousands of instances, add hosts in pools rather than increasing overcommit without measurement.

The 60-VM memory-only estimate per host is a conservative planning ceiling based on 8 GB per VM and a 32-GB host reserve. Actual SIS density should be lower if peak working sets, HA failover and migration headroom require it. A separate pool also allows the university to scale research compute without exposing student-data workloads to external tenant scheduling and kernel risks.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT

A. Impact analysis

The new requirement changes the threat model from mostly internal university departments to a genuinely multi-tenant environment containing external collaborators. The assumption that tenants are trusted is no longer valid. Spark/Hadoop jobs run for hours, generate sustained disk and network traffic and may create large shuffle bursts. Registration-week demand is three times normal, so the same CPU and I/O headroom that was adequate at ordinary load may be exhausted. Live migration adds traffic and temporary CPU/memory work precisely when maintenance is occurring. Regulation now requires demonstrable separation of student records, not merely a policy statement.

B. Evaluation of the original design

A single shared VM pool is not adequate as the final design. Even if hypervisor isolation is strong, shared CPU queues, storage backends, virtual switches and management credentials create noisy-neighbour and lateral-movement risks. The original fair-share network model $T_i = \eta B/N$ shows that adding long-running data jobs increases N and reduces the bandwidth available to interactive services. At three times normal

offered load, the system must have sufficient reserved capacity or it will violate the 10% degradation target. A shared pool cannot guarantee that simply by increasing vCPU overcommit.

C. Adaptation / redesign

Redesign into separate resource and trust zones:

SIS protected pool: dedicated Type-1 hosts, no external tenant placement, reserved CPU and RAM, dedicated or logically isolated storage volumes, strict database allow-lists and separate administrative roles.

LMS and examination pool: Type-1 VMs with resource reservations and independent network policies; scale-out web tiers may use containers only when their data access is constrained.

External research pool: micro-VMs for collaborator-controlled Spark/Hadoop workers, isolated virtual networks, tenant-specific storage, quotas and egress controls. No direct route to SIS databases.

Data plane: separate VLAN/VXLAN segments and firewall policy; dedicated NIC queues or SR-IOV where supported and validated; rate limits and QoS protect SIS traffic from shuffle traffic.

Migration plane: dedicated migration network, authenticated hosts, encrypted migration state, strict destination-host allow-list and audit logs. Do not migrate SIS VMs to an external/research pool.

Control plane: MFA, least privilege, signed images, central secrets management, immutable audit logs, vulnerability scanning and approved change windows.

This redesign lowers the density available to each pool because capacity is reserved and workloads cannot freely share all hosts. However, it reduces the attack surface between external workloads and student records, limits noisy-neighbour effects and makes compliance evidence easier to demonstrate. SR-IOV can reduce virtual-switch overhead for data-intensive research traffic, but it reduces some flexibility in live migration and must be tested with the chosen hypervisor and NIC.

D. Evidence-based defence

The decision follows from the model and the changed requirements. The host has 60 usable cores and 480 GB workload memory under the stated reserve assumptions. At a 3× registration peak, admission control must ensure the SIS pool has enough reserved CPU and memory to meet its target without relying on external-pool idle capacity. For each NIC, bandwidth must be partitioned or policed so research shuffle traffic cannot consume the SIS allocation. The fair-share expression $\eta B/N$ explains why a shared, uncontrolled network is unsafe for latency-sensitive registration.

The proposed architecture accepts a reduction in maximum density in exchange for demonstrable isolation. The most important residual risks are hypervisor zero-days, compromised management credentials, shared hardware side channels, storage misconfiguration and human error. These are addressed through patching, MFA, dedicated pools, encryption, micro-segmentation, signed images and continuous monitoring. Before production approval, the university must run the Task 8 test plan, populate Task 9 with actual results, and verify Task 10 error calculations under both normal and 3× peak load. Until that evidence exists, the design is analytically justified but not experimentally certified.

CONCLUSION

EduCloud should adopt a hybrid, pool-based virtualization strategy: hardened Type-1 VMs for SIS and other critical systems, containers for trusted stateless components, and micro-VMs for external research workloads. Quantitative modelling identifies memory as the immediate conservative VM-density limit (60 eight-GB VMs per host after a 32-GB reserve), while CPU overcommit, storage queueing and network contention determine whether the service-level target is actually met. Security controls must be applied across the hypervisor, host kernel, image pipeline, virtual network, storage and management plane. The final design prioritises student-data isolation and registration availability while retaining efficient scaling for research workloads.

REFERENCES

1. NIST. (2011). Guide to Security for Full Virtualization Technologies (SP 800-125). National Institute of Standards and Technology.

2. NIST. (2013). Security Recommendations for Server-based Hypervisor Platforms (SP 800-125A). National Institute of Standards and Technology.

3. NIST. (2017). Application Container Security Guide (SP 800-190). National Institute of Standards and Technology.

4. Linux Kernel Documentation. Control Group v2 (cgroup v2) documentation. https://docs.kernel.org/admin-guide/cgroup-v2.html

5. QEMU Project. QEMU System Emulation User's Guide. https://www.qemu.org/docs/

6. KVM Project. Kernel-based Virtual Machine documentation. https://www.linux-kvm.org/

7. Docker Documentation. Resource constraints and security. https://docs.docker.com/

8. Firecracker. Secure and lightweight virtualization for serverless computing. https://firecracker-microvm.github.io/

9. Kata Containers. Architecture and documentation. https://katacontainers.io/

10. Linux man-pages project. namespaces(7), cgroups(7), seccomp(2) and capabilities(7). https://man7.org/linux/man-pages/

Note on references and evidence: These references provide foundational technical documentation and security guidance. The assignment brief requests experimental evidence; record actual software versions, configurations, commands, raw output, screenshots and repeated measurements in the final submitted copy. Do not describe illustrative calculations or blank templates as experiments that have already been performed.